RESIDÊNCIA TECNOLÓGICA EM SISTEMAS EMBARCADOS

EmbarcaTech — Turma 2026

VitalKare

Plataforma Hospitalar Inteligente de Monitoramento à Beira do Leito

Documento de Arquitetura e Viabilidade Técnica

Entregável Inicial — Fase de Concepção

Hardware:	KIIRA V2 (ESP32-S3)
Firmware:	ESP-IDF 5.x + FreeRTOS
Backend:	FastAPI (Python 3.11)
Dashboard:	React 18 + TypeScript
Equipe:	*Nathan Linhares Dias Malheiros*
	João Pedro Queiroz Rodrigues
Data:	Maio de 2026

Dados para Ficha Catalográfica

Título:	VitalKare — Plataforma Hospitalar Inteligente
Subtítulo:	Documento de Arquitetura e Viabilidade Técnica
Autores:	Nathan Linhares Dias Malheiros João Pedro Queiroz Rodrigues
Ano:	2026
Páginas:	61
Palavras-chave:	ESP32-S3, FreeRTOS, IoT, TCP/IP, Firmware Embarcado

Resumo

O **VitalKare** é uma plataforma embarcada de monitoramento hospitalar à beira do leito, desenvolvida como projeto de encerramento da Residência Tecnológica EmbarcaTech em Sistemas Embarcados. O sistema integra coleta de sinais vitais em tempo real, gerenciamento de alertas de chamada e emergência, gravação e transcrição de áudio por inteligência artificial, ajuste motorizado de posição de leito e comunicação com um dashboard centralizado de enfermagem.

A arquitetura de hardware é centralizada no microcontrolador **ESP32-S3** (plataforma KIIRA V2), escolhido por seu desempenho dual-core Xtensa LX7, periféricos nativos para I2C, I2S, SPI e PWM, além de conectividade Wi-Fi 802.11 b/g/n integrada. Os sensores principais compreendem o **HDC1080** para temperatura e umidade ambiente, o **MAX30102/MAX30105** para oximetria de pulso e frequência cardíaca, o **MPU6050** para medição de inclinação do leito, microfone PDM via barramento I2S e display OLED SSD1306. O controle mecânico do leito é realizado por servo motor comandado por sinal PWM.

O firmware é estruturado sobre o sistema operacional de tempo real **FreeRTOS**, com arquitetura orientada a eventos mediada por filas e semáforos. Os botões de chamada e emergência são tratados exclusivamente por rotinas de serviço de interrupção (*Interrupt Service Routines* — ISR), garantindo latência de resposta sub-milissegundo. A comunicação de rede adota **TCP persistente** como protocolo primário, após avaliação comparativa com MQTT, privilegiando menor sobrecarga de protocolo, determinismo de entrega e adequação ao ambiente de LAN hospitalar controlado.

O backend é implementado em **FastAPI** (Python 3.11) com armazenamento em **PostgreSQL 16 + TimescaleDB**, cache em **Redis 7** e armazenamento de mídia em **MinIO** (S3-compatível). A transcrição de áudio é realizada localmente pelo modelo **Faster-Whisper** (PT-BR), preservando a privacidade dos dados dos pacientes. O dashboard em **React 18 + TypeScript** recebe atualizações em tempo real via WebSocket.

Este documento formaliza a arquitetura completa, o mapeamento de hardware, os protocolos de comunicação, a viabilidade técnica e os modelos de dados do sistema,

constituindo o entregável inicial obrigatório do projeto.

Palavras-chave: ESP32-S3; FreeRTOS; IoT Hospitalar; Monitoramento de Sinais Vitais; TCP/IP Embarcado; Transcrição de Áudio; FastAPI; React; Sistemas de Tempo Real.

Abstract

VitalKare is an embedded bedside hospital monitoring platform developed as the capstone project of the EmbarcaTech Technology Residency in Embedded Systems. The system integrates real-time vital signs collection, call and emergency alert management, AI-powered audio recording and transcription, motorized bed positioning, and centralized nursing dashboard communication.

The hardware architecture centers on the **ESP32-S3** microcontroller (KIIRA V2 platform), chosen for its dual-core Xtensa LX7 performance, native I2C, I2S, SPI, and PWM peripherals, and integrated 802.11 b/g/n Wi-Fi. Core sensors include the **HDC1080** for ambient temperature and humidity, **MAX30102/MAX30105** for pulse oximetry and heart rate, **MPU6050** for bed tilt measurement, PDM microphone via I2S bus, and SSD1306 OLED display. Bed mechanical control is performed by a PWM-driven servo motor.

Firmware is built on **FreeRTOS**, with an event-driven architecture mediated by queues and semaphores. Call and emergency buttons are handled exclusively by hardware ISRs, ensuring sub-millisecond response latency. Network communication employs **persistent TCP** as the primary protocol, selected after comparative evaluation with MQTT due to lower protocol overhead, delivery determinism, and suitability to the controlled hospital LAN environment.

Keywords: ESP32-S3; FreeRTOS; Hospital IoT; Vital Signs Monitoring; Embedded TCP/IP; Audio Transcription; FastAPI; React; Real-Time Systems.

Sumário

Resumo **2**

Abstract **4**

Lista de Figuras **8**

Lista de Tabelas **9**

Lista de Abreviaturas e Siglas **10**

1 Introdução **12**

1.1 Contexto e Motivação 12

1.2 O Problema Específico 13

1.3 Solução Proposta 13

1.4 Organização do Documento 14

2 Objetivos **15**

2.1 Objetivo Geral 15

2.2 Objetivos Específicos 15

3 Visão Geral do Sistema **17**

3.1 Arquitetura em Camadas 17

3.2 Fluxo Funcional Principal 17

3.3 Topologia de Rede 18

4 Mapeamento de Hardware e Componentes **20**

4.1 Identificação dos Componentes 20

4.1.1 ESP32-S3 — Microcontrolador Principal (KIIRA V2) 20

4.1.2 HDC1080 — Sensor de Temperatura e Umidade Relativa 22

4.1.3 MAX30102/MAX30105 — Sensor de Oximetria e Frequência Cardíaca 23

4.1.4 MPU6050 — Acelerômetro e Giroscópio (IMU) 23

4.1.5 SSD1306 — Display OLED 128×64 24

4.1.6 Microfone PDM — Captura de Áudio . . . 24
4.1.7 Servo Motor . . . 25
4.1.8 Botões de Chamada e Emergência . . . 25
4.1.9 Módulo SD Card . . . 26
4.1.10 Interface Wi-Fi . . . 26
4.2 Tabela de Mapeamento de GPIOs . . . 26
4.3 Barramentos Internos e Justificativa de Escolha . . . 28
4.3.1 Barramento I2C . . . 28
4.3.2 Barramento I2S para Microfone PDM . . . 29
4.3.3 PWM — LEDC para Servo Motor . . . 29
4.3.4 Interrupções GPIO para Botões . . . 29

5 Diagrama de Blocos do Sistema 31
5.1 Diagrama de Blocos de Hardware . . . 31
5.2 Diagrama de Blocos de Software . . . 31

6 Arquitetura de Firmware 33
6.1 Modelo de Execução: FreeRTOS . . . 33
6.2 Hierarquia de Tarefas . . . 33
6.3 Tratamento de Interrupções (ISR) . . . 34
6.4 Pipeline de Áudio . . . 35
6.5 Controle de Posicionamento (MPU6050 + Servo) . . . 36
6.6 Watchdog e Confiabilidade . . . 37

7 Especificação do Protocolo de Comunicação e IoT 38
7.1 Avaliação e Seleção de Protocolo . . . 38
7.1.1 MQTT — Avaliação e Descarte como Protocolo Primário . . . 38
7.1.2 TCP — Protocolo Selecionado . . . 39
7.2 Arquitetura TCP . . . 40
7.2.1 Modelo de Conexão . . . 40
7.2.2 Framing de Pacotes . . . 40
7.2.3 Estratégia de ACK/NACK . . . 41
7.3 Estrutura de Dados — Payloads JSON . . . 41
7.3.1 Pacote de Telemetria . . . 41
7.3.2 Pacote de Chamada Simples (com Áudio) . . . 42
7.3.3 Pacote de Emergência . . . 42

7.4 Fluxo de Áudio — Detalhamento 43

8 Dashboard e Servidor **45**

8.1 Arquitetura do Backend 45

8.2 Camada de Dados 46

8.3 Dashboard React 18 46

8.4 Interação Operacional da Equipe de Enfermagem 47

9 Modelagem 3D dos Gabinetes **48**

9.1 Gabinete Principal — Beira do Leito 48

9.1.1 Especificações de Design 48

9.1.2 Posicionamento do Acelerômetro 48

9.2 Acessório Sensor Cardíaco — Oxímetro de Dedo 49

9.2.1 Requisitos de Design 49

9.3 Acoplamento Mecânico do Servo Motor 50

10 Viabilidade Técnica **51**

10.1 Viabilidade de Hardware 51

10.2 Viabilidade de Firmware 51

10.2.1 Capacidade de Memória 52

10.2.2 Requisitos de Tempo Real 52

10.3 Viabilidade de Rede 53

10.4 Viabilidade de Escalabilidade 53

10.5 Riscos Técnicos e Mitigações 54

11 Conclusão **55**

11.1 Trabalhos Futuros 55

11.2 Considerações Finais 56

Referências Bibliográficas **57**

A Estrutura do Repositório **59**

B Guia de Provisionamento do Firmware **61**

Lista de Figuras

3.1 Topologia de rede do sistema VitalKare em ambiente de LAN hospitalar. 19

5.1 Diagrama de blocos do sistema VitalKare. 31
5.2 Organização em tarefas FreeRTOS do firmware VitalKare. As setas indicam fluxo de dados via filas e eventos. 32

6.1 Pipeline completo de captura, gravação, transmissão e transcrição de áudio de emergência. 36

9.1 Render 3D do gabinete principal VitalKare (a ser inserido após modelagem CAD). 49
9.2 Render 3D do acessório de oxímetria de pulso VitalKare (a ser inserido após modelagem CAD). 50

Lista de Tabelas

4.1 Especificações do microcontrolador ESP32-S3 (KIIRA V2). 21
4.2 Especificações do sensor HDC1080 (Texas Instruments). 22
4.3 Especificações do sensor MAX30102/MAX30105 (Maxim Integrated). . 23
4.4 Especificações do MPU6050 (InvenSense/TDK). 23
4.5 Especificações do display OLED SSD1306. 24
4.6 Especificações do microfone PDM (MEMS). 24
4.7 Especificações do servo motor de posicionamento. 25
4.8 Especificações dos botões de interação do paciente. 25
4.9 Especificações do módulo de cartão SD. 26
4.10 Especificações da interface Wi-Fi do ESP32-S3. 26
4.11 Mapeamento de GPIOs do ESP32-S3 — VitalKare KIIRA V2. 27

6.1 Tarefas FreeRTOS do firmware VitalKare. 34

7.1 Estrutura do cabeçalho de pacote TCP (framing binário). 41

8.1 Principais entidades de dados do sistema VitalKare. 46

9.1 Requisitos de design do gabinete principal VitalKare. 48
9.2 Requisitos de design do acessório de oxímetria de dedo. 49

10.1 Análise de utilização de memória do firmware VitalKare. 52
10.2 Análise de largura de banda de rede do sistema VitalKare. 53
10.3 Matriz de riscos técnicos e estratégias de mitigação. 54

Lista de Abreviaturas e Siglas

Sigla	Descrição
ACK	*Acknowledgement* — Confirmação de Pacote
ADC	*Analog-to-Digital Converter* — Conversor Analógico-Digital
AI	*Artificial Intelligence* — Inteligência Artificial
ADPCM	*Adaptive Differential Pulse-Code Modulation*
API	*Application Programming Interface*
BPM	Batimentos Por Minuto
DMA	*Direct Memory Access* — Acesso Direto à Memória
FreeRTOS	*Free Real-Time Operating System*
GPIO	*General Purpose Input/Output* — Porta de Uso Geral
I2C	*Inter-Integrated Circuit* — Barramento Serial Síncrono
I2S	*Inter-IC Sound* — Barramento de Áudio Digital
IoT	*Internet of Things* — Internet das Coisas
ISR	*Interrupt Service Routine* — Rotina de Serviço de Interrupção
ISR	*Interrupt Service Routine*
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
LAN	*Local Area Network* — Rede Local
MCU	*Microcontroller Unit* — Microcontrolador
MQTT	*Message Queuing Telemetry Transport*
NVS	*Non-Volatile Storage* — Armazenamento Não-Volátil
OLED	*Organic Light-Emitting Diode*
OTA	*Over-The-Air* — Atualização Remota de Firmware
PCB	*Printed Circuit Board* — Placa de Circuito Impresso
PDM	*Pulse-Density Modulation*
PSRAM	*Pseudo-Static Random-Access Memory*
PWM	*Pulse Width Modulation* — Modulação por Largura de Pulso
QoS	*Quality of Service* — Qualidade de Serviço
RBAC	*Role-Based Access Control*

Sigla	**Descrição**
REST	*Representational State Transfer*
SPI	*Serial Peripheral Interface*
SpO2	Saturação de Oxigênio no Sangue
TCP	*Transmission Control Protocol*
TLS	*Transport Layer Security*
UDP	*User Datagram Protocol*
UI	*User Interface* — Interface de Usuário
WDT	*Watchdog Timer*
Wi-Fi	*Wireless Fidelity* — IEEE 802.11

Capítulo 1

Introdução

1.1 Contexto e Motivação

O ambiente hospitalar contemporâneo de alta complexidade enfrenta pressões crescentes sobre a eficiência operacional das equipes de enfermagem, a segurança dos pacientes internados e a qualidade da experiência do cuidado prestado. Em enfermarias com elevada taxa de ocupação, o monitoramento contínuo e individualizado de cada leito representa um desafio logístico significativo, frequentemente resultando em atrasos na resposta a intercorrências, sobrecarga profissional e riscos desnecessários à integridade clínica dos pacientes.

O sistema de chamada de enfermeira convencional, baseado em um único botão de pressão que aciona um sinal sonoro ou luminoso na central do posto de enfermagem, constitui a infraestrutura predominante em hospitais brasileiros. Embora funcional em sua concepção original, este modelo apresenta limitações críticas que se tornam progressivamente mais evidentes à medida que a complexidade clínica das enfermarias aumenta. O profissional de enfermagem que recebe o acionamento não dispõe, no momento do alerta, de qualquer informação qualitativa sobre a natureza do evento: se trata-se de uma solicitação de conforto rotineira, de uma queda, de uma crise respiratória ou de uma parada cardiorrespiratória. Esta ausência de contexto pré-chegada implica que o profissional deve deslocar-se ao quarto sem o equipamento, a preparação mental ou a prioridade de atendimento adequados à gravidade real da situação.

Paralelamente, o conforto e a segurança biomecânica dos pacientes acamados por longos períodos dependem criticamente do ajuste periódico da posição do leito. A prevenção de lesões por pressão (escaras), uma das complicações hospitalares mais custosas e debilitantes para o paciente, exige que os ângulos de decúbito sejam modificados regularmente. Na infraestrutura atual, este ajuste depende inteiramente da presença física de um técnico de enfermagem que manipule a manivela mecânica do leito — um processo que, por depender de disponibilidade humana, frequentemente ocorre com

intervalos irregulares ou insuficientes.

1.2 O Problema Específico

Este projeto foi motivado por uma demanda concreta formulada pela direção de inovação e enfermagem de um hospital de alta complexidade, que identificou cinco pontos críticos de ineficiência:

1. **Ausência de triagem pré-chegada:** O sistema de chamada vigente não comunica gravidade, impossibilitando a priorização de atendimento pela equipe de enfermagem.
2. **Falta de contexto de emergência:** Em situações de risco imediato à vida, a equipe chega ao quarto sem saber o que encontrará, retardando a mobilização de recursos adequados.
3. **Monitoramento passivo de sinais vitais:** A ausência de telemetria contínua e automatizada de frequência cardíaca e temperatura ambiente força o profissional a realizar rondas físicas periódicas para coletar informações que poderiam ser transmitidas automaticamente.
4. **Dependência humana para ajuste de leito:** O reposicionamento do paciente para prevenção de escaras requer intervenção manual frequente, sobrecarregando a equipe técnica em plantões de alta demanda.
5. **Indisponibilidade de informação histórica:** Alertas, sinais vitais e eventos clínicos não são registrados sistematicamente de forma associada ao leito, dificultando auditorias de qualidade e continuidade do cuidado entre turnos.

1.3 Solução Proposta

O **VitalKare** propõe uma abordagem integrada de automação embarcada para endereçar esses cinco pontos críticos. Trata-se de um dispositivo IoT fixado à beira do leito, desenvolvido sobre o microcontrolador ESP32-S3 (plataforma KIIRA V2), que unifica em uma única plataforma de baixo custo e alta confiabilidade:

- Monitoramento contínuo de frequência cardíaca e saturação de oxigênio via sensor de oximetria de pulso conectado ao dedo do paciente;

- Monitoramento de temperatura e umidade do ambiente do quarto;
- Interface de chamada de dois níveis: **Chamada Simples** (botão A) e **Emergência com Áudio** (botão B);
- Gravação de áudio ambiente durante o pressionamento do botão de emergência e envio assíncrono para transcrição automática por IA;
- Ajuste motorizado da inclinação do leito via servo motor, guiado por leitura de ângulo do acelerômetro interno (MPU6050);
- Exibição local em display OLED de sinais vitais, status de alertas e informações do paciente;
- Transmissão em tempo real de todos os dados para um servidor central via TCP persistente sobre Wi-Fi 802.11;
- Dashboard web centralizado no posto de enfermagem, com visualização em tempo real, alertas visuais escalonados por gravidade e transcrições de áudio de emergências.

1.4 Organização do Documento

Este documento está estruturado nos seguintes capítulos: o Capítulo 2 formaliza os objetivos gerais e específicos; o Capítulo 3 apresenta a visão geral da arquitetura do sistema; o Capítulo 4 detalha o mapeamento de hardware e periféricos; o Capítulo 5 apresenta o diagrama de blocos do sistema; o Capítulo 6 descreve a arquitetura de firmware; o Capítulo 7 especifica os protocolos de comunicação e IoT; o Capítulo 8 descreve a arquitetura do servidor e dashboard; o Capítulo 9 discute a modelagem 3D dos gabinetes; o Capítulo 10 avalia a viabilidade técnica do projeto; e o Capítulo 11 apresenta as conclusões e perspectivas futuras.

Capítulo 2

Objetivos

2.1 Objetivo Geral

Desenvolver uma plataforma embarcada de monitoramento hospitalar à beira do leito, baseada no microcontrolador ESP32-S3, capaz de coletar sinais vitais em tempo real, gerenciar alertas de chamada e emergência com captura e transcrição de áudio por inteligência artificial, controlar a inclinação mecânica do leito por servo motor e transmitir todos os dados para um dashboard centralizado de enfermagem via rede TCP/IP local.

2.2 Objetivos Específicos

OE1. Monitoramento de Sinais Vitais: Implementar a leitura contínua e precisa de frequência cardíaca (BPM) e saturação de oxigênio (SpO2) por meio do sensor MAX30102/MAX30105, além de temperatura e umidade ambiente via HDC1080, com transmissão periódica ao servidor em janelas de 5 segundos.

OE2. Sistema de Chamada Dual-Level: Implementar dois botões físicos — Chamada Simples (botão A) e Emergência (botão B) — tratados por interrupções de hardware (ISR), garantindo resposta imediata independentemente do estado das demais tarefas do firmware.

OE3. Captura e Transmissão de Áudio de Emergência: Implementar o pipeline completo de gravação de áudio PDM (16 kHz, mono, armazenado em PSRAM), com início no pressionamento do botão de emergência, encerramento no soltar do botão ou ao atingir 10 segundos, e transmissão assíncrona do buffer para o servidor via TCP.

OE4. Transcrição Automática por IA: Integrar o serviço de transcrição Faster-Whisper (modelo small, PT-BR, self-hosted) no backend, exibindo a trans-

crição do áudio de emergência no dashboard do posto de enfermagem antes da chegada do profissional ao quarto.

OE5. Controle de Posicionamento por Servo Motor: Implementar a leitura do ângulo de inclinação do ESP32-S3 via MPU6050 (acelerômetro + giroscópio) e controlar um servo motor por sinal PWM para ajustar automaticamente a inclinação do leito para a posição alvo, simulando a automação da manivela mecânica.

OE6. Interface Local com Display OLED: Manter uma interface de usuário atualizada em tempo real no display SSD1306 (128×64 px), exibindo identificação do paciente, BPM, SpO2, temperatura, status de conectividade Wi-Fi e indicação visual de alertas ativos.

OE7. Arquitetura de Firmware Concorrente: Estruturar o firmware sobre FreeRTOS com tarefas independentes para cada subsistema (sensores, áudio, rede, display, botões, servo), comunicação inter-task via filas e semáforos, e watchdog de hardware para garantia de operação contínua.

OE8. Comunicação TCP Persistente: Implementar cliente TCP persistente com estratégia de reconexão exponencial, pacotes binarizados com cabeçalho de comprimento, heartbeat periódico e upload assíncrono de áudio sem bloqueio da telemetria principal.

OE9. Dashboard Centralizado em Tempo Real: Desenvolver interface web React 18 que receba eventos via WebSocket e exiba o status de todos os leitos, alertas visuais escalonados por gravidade, transcrições de áudio e histórico de eventos por leito.

OE10. Modelagem 3D dos Gabinetes: Projetar dois gabinetes impressos em 3D: o gabinete principal para fixação à beira do leito (com aberturas para display, microfone e botões) e o acessório ergonômico de oximetria de dedo (tipo clip hospitalar com isolamento óptico).

Capítulo 3

Visão Geral do Sistema

3.1 Arquitetura em Camadas

O VitalKare é organizado em quatro camadas funcionais distintas, cada uma com responsabilidades bem definidas e interfaces de comunicação padronizadas entre elas:

Camada 1 — Percepção (Sensing Layer): Responsável pela coleta de dados do ambiente físico. Inclui os sensores de temperatura, oximetria, aceleração e o microfone PDM. Todos os sensores se comunicam com o MCU por protocolos digitais (I2C, I2S), eliminando fontes de ruído analógico e permitindo distâncias de cabo maiores.

Camada 2 — Processamento Embarcado (Edge Layer): O ESP32-S3 executa o firmware FreeRTOS, realizando fusão de dados dos sensores, processamento do pipeline de áudio, controle de servo, gestão de alertas e comunicação TCP. É nesta camada que ocorre toda a lógica de tempo real do sistema.

Camada 3 — Transporte e Mensageria (Network Layer): Wi-Fi 802.11 b/g/n operando na LAN hospitalar, com socket TCP persistente ao servidor. O ESP32-S3 atua como cliente TCP, enquanto o servidor FastAPI mantém conexões persistentes por leito, permitindo comunicação bidirecional.

Camada 4 — Serviços e Apresentação (Application Layer): Backend FastAPI com banco de dados PostgreSQL/TimescaleDB, serviço de transcrição Faster-Whisper, armazenamento de áudio MinIO e dashboard React 18 acessível pelos postos de enfermagem via navegador web.

3.2 Fluxo Funcional Principal

O funcionamento normal do sistema segue o seguinte fluxo de operações concorrentes:

1. A cada 5 segundos, a tarefa de sensores lê BPM, SpO2 e temperatura e enfileira um pacote de telemetria para a tarefa de rede.
2. A tarefa de rede transmite o pacote via socket TCP ao servidor, que persiste os dados no banco de dados TimescaleDB e propaga a atualização ao dashboard via WebSocket.
3. A tarefa de display atualiza o OLED com os dados mais recentes dos sensores e o status de conectividade.
4. A tarefa do servo motor lê continuamente o ângulo do MPU6050 e ajusta o duty cycle do PWM para manter a inclinação do leito na posição alvo configurada.
5. Ao pressionar o botão A (Chamada Simples), a ISR correspondente sinaliza a fila de eventos de alta prioridade. A tarefa de rede envia imediatamente um pacote de alerta ao servidor, que dispara notificação visual no dashboard.
6. Ao pressionar o botão B (Emergência), a ISR de emergência é disparada, iniciando simultaneamente: (a) envio imediato de alerta de emergência ao servidor e (b) início de gravação de áudio PDM no buffer PSRAM.
7. Ao soltar o botão B (ou após 10 segundos), a gravação é encerrada, o buffer é fechado e enfileirado para upload assíncrono via TCP ao servidor.
8. O servidor recebe o áudio, armazena em MinIO e submete ao serviço Faster-Whisper. A transcrição resultante é exibida no dashboard antes da chegada do profissional ao quarto.

3.3 Topologia de Rede

A Figura 3.1 ilustra a topologia de rede do sistema VitalKare em ambiente hospitalar.

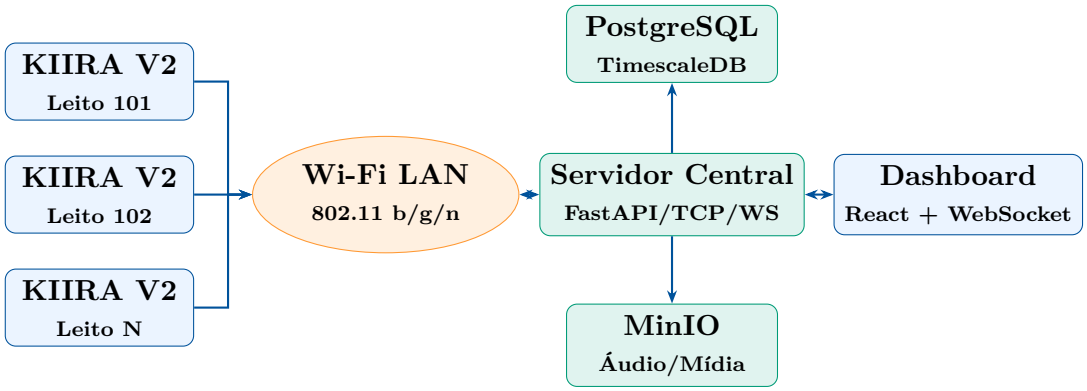


Figura 3.1: Topologia de rede do sistema VitalKare em ambiente de LAN hospitalar.

Capítulo 4

Mapeamento de Hardware e Componentes

4.1 Identificação dos Componentes

Esta seção documenta todos os componentes eletrônicos que compõem o nó embarcado VitalKare, com suas especificações técnicas relevantes para o desenvolvimento do firmware e para a compreensão das decisões de projeto.

4.1.1 ESP32-S3 — Microcontrolador Principal (KIIRA V2)

O microcontrolador ESP32-S3, fabricado pela Espressif Systems, é o coração do sistema VitalKare. A plataforma KIIRA V2 utiliza este chip em conjunto com 8 MB de PSRAM QSPI externa, essencial para o armazenamento do buffer de áudio (até 160 KB por gravação de 10 s a 16 kHz/16-bit/mono).

Tabela 4.1: Especificações do microcontrolador ESP32-S3 (KIIRA V2).

Parâmetro	**Valor**
Núcleos CPU	2× Xtensa LX7 @ 240 MHz
Flash interna	8 MB
PSRAM externa	8 MB QSPI (Octal)
Wi-Fi	802.11 b/g/n (2,4 GHz)
Bluetooth	BLE 5.0
GPIO disponíveis	Até 45 pinos (KIIRA V2: 38 expostos)
ADC	2× SAR ADC 12-bit (20 canais)
DAC	2× DAC 8-bit
I2C	2× I2C (master/slave)
SPI	4× SPI (QSPI)
I2S	2× I2S full-duplex
PWM (LEDC)	8 canais, resolução de 1–20 bits
UART	3× UART
USB OTG	USB 2.0 Full Speed nativo
Tensão de operação	3,3 V
Consumo ativo	~250–350 mA (Wi-Fi TX ativo)
Consumo sleep	~10–20 µA (deep sleep)
SDK	ESP-IDF 5.x (FreeRTOS integrado)

4.1.2 HDC1080 — Sensor de Temperatura e Umidade Relativa

Tabela 4.2: Especificações do sensor HDC1080 (Texas Instruments).

Parâmetro	Valor
Função	Medição de temperatura e umidade relativa
Interface	I2C (endereço 0x40)
Tensão de operação	2,7 V a 5,5 V (nível lógico 3,3 V)
Faixa de temperatura	-40 °C a +125 °C
Precisão de temperatura	±0,2 °C (típico)
Faixa de umidade	0 % a 100 % UR
Precisão de umidade	±2 % UR (típico)
Resolução	14-bit (temperatura e umidade)
Tempo de conversão	6,5 ms (14-bit)
Consumo de corrente	1,25 µA (modo sleep) / 710 µA (ativo)
Finalidade no sistema	Monitoramento ambiental do quarto (transmissão de telemetria)

4.1.3 MAX30102/MAX30105 — Sensor de Oximetria e Frequência Cardíaca

Tabela 4.3: Especificações do sensor MAX30102/MAX30105 (Maxim Integrated).

Parâmetro	Valor
Função	Oximetria de pulso (SpO2) e frequência cardíaca (BPM)
Interface	I2C (endereço 0x57)
Tensão de operação	1,8 V (digital) / 3,3 V a 5,0 V (LEDs)
LEDs emissores	Vermelho (660 nm) + Infravermelho (880 nm)
Fotodetector	Fotodiodo de alta sensibilidade
ADC interno	18-bit
Taxa de amostragem	50 Hz a 3200 Hz (configurável)
Precisão SpO2	±2 % (com algoritmo de processamento)
Corrente dos LEDs	0 mA a 51 mA (programável)
FIFO interno	32 amostras
Finalidade	Sensor externo (acessório de dedo) para telemetria clínica
Observações	Requer algoritmo de extração de pico (firmware side)

4.1.4 MPU6050 — Acelerômetro e Giroscópio (IMU)

Tabela 4.4: Especificações do MPU6050 (InvenSense/TDK).

Parâmetro	Valor
Função	Medição de aceleração (3 eixos) e velocidade angular (3 eixos)
Interface	I2C (endereço 0x68 ou 0x69)
Tensão de operação	2,375 V a 3,46 V
Faixa acelerômetro	±2g / ±4g / ±8g / ±16g (selecionável)
Faixa giroscópio	±250 / ±500 / ±1000 / ±2000 °/s
ADC interno	16-bit
DMP interno	*Digital Motion Processor* integrado
Taxa de saída	Até 1 kHz (acelerômetro), 8 kHz (giroscópio)
Consumo de corrente	3,9 mA (modo normal)
Finalidade	Cálculo do ângulo de inclinação do dispositivo/leito
Observações	Ângulo calculado via arco-tangente sobre eixos X e Z

4.1.5 SSD1306 — Display OLED 128×64

Tabela 4.5: Especificações do display OLED SSD1306.

Parâmetro	Valor
Função	Interface visual local (informações clínicas e alertas)
Interface	I2C (endereço 0x3C ou 0x3D)
Resolução	128 × 64 pixels
Tamanho típico	0,96 polegadas (diagonal)
Tensão de operação	3,3 V ou 5 V (módulo com regulador)
Consumo	~20 mA (operação normal)
Profundidade de cor	1-bit (monocromático, branco sobre fundo preto)
Ângulo de visão	Próximo a 160°
Telas implementadas	7 (normal, alerta, emergência, configuração, shutdown, etc.)
Finalidade	Exibição de BPM, SpO2, temperatura, status de rede e alertas

4.1.6 Microfone PDM — Captura de Áudio

Tabela 4.6: Especificações do microfone PDM (MEMS).

Parâmetro	Valor
Função	Captura de áudio ambiente durante eventos de emergência
Interface	I2S (PDM — *Pulse-Density Modulation*)
Taxa de amostragem	16.000 Hz (16 kHz)
Resolução	16-bit PCM (após decimação no ESP32-S3)
Canais	Mono (1 canal)
Frequência de clock PDM	1,024 MHz
Tensão de operação	1,8 V a 3,3 V
Razão sinal-ruído	~61 dB (típico para MEMS PDM)
Duração máxima	10 segundos (160 KB em PSRAM, formato PCM raw)
Finalidade	Gravação de emergências para transcrição por IA
Observações	Buffer alocado em PSRAM para não consumir RAM interna

4.1.7 Servo Motor

Tabela 4.7: Especificações do servo motor de posicionamento.

Parâmetro	Valor
Função	Controle de inclinação mecânica do leito (simulação de manivela)
Interface	PWM (sinal digital 50 Hz, duty cycle variável)
Tensão de operação	5 V (alimentação separada, não do ESP32)
Ângulo de rotação	0° a 180°
Pulso PWM	500 µs (0°) a 2500 µs (180°)
Torque típico	~2,5 kg · cm a 5 V (modelo padrão SG90 ou similar)
Frequência de sinal	50 Hz (período de 20 ms)
Resolução LEDC	16-bit (65536 divisões de duty cycle)
Finalidade	Ajuste automático da inclinação do leito para prevenção de escaras
Observações	Alimentação independente obrigatória; GND compartilhado com ESP32

4.1.8 Botões de Chamada e Emergência

Tabela 4.8: Especificações dos botões de interação do paciente.

Parâmetro	Valor
Quantidade	2 botões físicos (Botão A: Chamada / Botão B: Emergência)
Tipo	*Push button* com resistor de pull-up interno (GPIO_PULLUP)
Configuração GPIO	Entrada digital, ativo em nível baixo (pressão = GND)
Tratamento	Interrupção de hardware (ISR) em borda de descida e subida
Debounce	Por software (janela de 50 ms na ISR)
Tensão de operação	3,3 V
Sinalização visual	LEDs de indicação correspondentes (verde/vermelho)
Finalidade A	Solicitar assistência de enfermagem (alerta visual no dashboard)
Finalidade B	Acionar emergência com gravação de áudio e alerta prioritário
Observações	ISR de alta prioridade; nunca deve ser mascarada por outras tarefas

4.1.9 Módulo SD Card

Tabela 4.9: Especificações do módulo de cartão SD.

Parâmetro	Valor
Função	Armazenamento local offline de eventos e logs
Interface	SDMMC (modo 4 bits) ou SPI
Tensão de operação	3,3 V
Formato de arquivo	JSON Lines (`.jsonl`) para logs estruturados
Capacidade suportada	MicroSD até 32 GB (FAT32)
Finalidade	Cache offline: armazena eventos e telemetria quando sem rede
Observações	Upload automático ao reconectar à rede Wi-Fi

4.1.10 Interface Wi-Fi

Tabela 4.10: Especificações da interface Wi-Fi do ESP32-S3.

Parâmetro	Valor
Padrão	IEEE 802.11 b/g/n
Frequência	2,4 GHz
Modo de operação	Station (STA) — cliente Wi-Fi
Antena	PCB embutida na placa KIIRA V2
Segurança	WPA2-PSK / WPA3 (configurable via NVS)
Gerenciamento	Wi-Fi Manager com backoff exponencial de reconexão
Credenciais	Armazenadas em NVS (provisionamento via script Python)
RSSI típico	-40 dBm a -70 dBm (ambiente hospitalar com boa cobertura)
Finalidade	Comunicação TCP persistente com servidor FastAPI

4.2 Tabela de Mapeamento de GPIOs

A Tabela 4.11 apresenta o mapeamento completo dos pinos GPIO do ESP32-S3 (plataforma KIIRA V2) utilizados no sistema VitalKare. Os pinos marcados com **INT** suportam interrupção externa de hardware. Os pinos marcados com **PWM** possuem capacidade de geração de sinal PWM via periférico LEDC.

Tabela 4.11: Mapeamento de GPIOs do ESP32-S3 — VitalKare KIIRA V2.

GPIO	Periférico	Direção	Protocolo	INT/PW	Observações
GPIO 8	HDC1080 SDA	Bidirecior	I2C (SDA)	—	Barramento I2C principal; pull-up 4,7 kΩ
GPIO 9	HDC1080 SCL	Saída	I2C (SCL)	—	Clock I2C 400 kHz (Fast Mode)
GPIO 8	SSD1306 SDA	Bidirecior	I2C (SDA)	—	Mesmo barramento I2C (endereço 0x3C)
GPIO 9	SSD1306 SCL	Saída	I2C (SCL)	—	Mesmo barramento I2C
GPIO 8	MPU6050 SDA	Bidirecior	I2C (SDA)	—	Mesmo barramento I2C (endereço 0x68)
GPIO 9	MPU6050 SCL	Saída	I2C (SCL)	—	Mesmo barramento I2C
GPIO 8	MAX30102 SDA	Bidirecior	I2C (SDA)	—	Mesmo barramento I2C (endereço 0x57)
GPIO 9	MAX30102 SCL	Saída	I2C (SCL)	—	Mesmo barramento I2C
GPIO 38	Microfone PDM	Entrada	I2S (Data)	—	I2S RX, modo PDM; canal esquerdo
GPIO 39	Microfone CLK	Saída	I2S (CLK)	—	Clock PDM a 1,024 MHz
GPIO 18	Servo Motor	Saída	PWM (LEDC)	**PWM**	Canal LEDC 0, 50 Hz, 16-bit
GPIO 0	Botão A (Chamada)	Entrada	GPIO Input	**INT**	ISR borda descida; pull-up interno
GPIO 35	Botão B (Emerg.)	Entrada	GPIO Input	**INT**	ISR borda descida e subida; pull-up
GPIO 36	LED Botão A	Saída	GPIO Output	—	Verde; indica chamada ativa
GPIO 37	LED Botão B	Saída	GPIO Output	—	Vermelho; indica emergência ativa

Continua na próxima página

GPIO	Periférico	Direção	Protocolo	INT/PW	Observações
GPIO 14	SD Card MOSI	Saída	SDMMC / SPI	—	Dados SD Card (modo SPI ou SDMMC)
GPIO 12	SD Card MISO	Entrada	SDMMC / SPI	—	Resposta SD Card
GPIO 13	SD Card CLK	Saída	SDMMC / SPI	—	Clock SD Card
GPIO 11	SD Card CS	Saída	SPI (CS)	—	Chip Select (modo SPI)
GPIO 3	UART TX (Debug)	Saída	UART0	—	Monitor serial de debug (921600 baud)
GPIO 1	UART RX (Debug)	Entrada	UART0	—	Recepção de comandos de debug
GPIO 20	MAX30102 INT	Entrada	GPIO Input	INT	Sinal de interrupção do sensor de pulso
GPIO 2	LED Status Wi-Fi	Saída	GPIO Output	—	Indicação de conectividade
GPIO 4	LDR (Luminosidade)	Entrada	ADC (canal 4)	—	Leitura de luminosidade ambiente (se disponível)

Nota sobre o barramento I2C: Todos os quatro sensores digitais (HDC1080, SSD1306, MPU6050 e MAX30102) compartilham o mesmo barramento I2C nos pinos GPIO 8 (SDA) e GPIO 9 (SCL), cada um identificado por seu endereço I2C único. O barramento opera a 400 kHz (modo Fast), com resistores de pull-up externos de 4,7 kΩ para 3,3 V.

4.3 Barramentos Internos e Justificativa de Escolha

4.3.1 Barramento I2C

O protocolo I2C (*Inter-Integrated Circuit*) foi selecionado para a comunicação com os sensores de medição e com o display OLED em razão de sua capacidade de multiplexação de múltiplos dispositivos em apenas dois fios (SDA e SCL), o que simplifica substancialmente o roteamento de cabos e a complexidade do PCB. Com apenas quatro

resistores de pull-up compartilhados, é possível endereçar independentemente os quatro periféricos digitais do sistema. A taxa de 400 kHz (modo Fast) é adequada para as taxas de atualização requeridas pelos sensores (o HDC1080 completa uma conversão em 6,5 ms; o MPU6050 opera a até 1 kHz internamente). O ESP-IDF fornece driver I2C nativo com suporte a transferências DMA, eliminando uso de CPU durante a coleta de dados.

4.3.2 Barramento I2S para Microfone PDM

O microfone MEMS utiliza a interface I2S no modo PDM (*Pulse-Density Modulation*), que é o padrão industrial para microfones digitais de baixo custo. O ESP32-S3 possui suporte nativo a I2S PDM com decimação interna de 64×, convertendo automaticamente o sinal PDM para PCM 16-bit no periférico de hardware, sem carga computacional na CPU. O uso de DMA para transferência do buffer I2S para a PSRAM é crítico para garantir que a gravação de áudio não interfira com as demais tarefas do firmware. A taxa de 16 kHz foi escolhida por ser o padrão mínimo recomendado pelo modelo Faster-Whisper para transcrição de qualidade aceitável em português.

4.3.3 PWM — LEDC para Servo Motor

O periférico LEDC (*LED Controller*) do ESP32-S3 é utilizado para gerar o sinal PWM de controle do servo motor. Embora nominalmente projetado para controle de LEDs, o LEDC é o periférico padrão para geração de PWM de uso geral no ESP-IDF. Configurado com frequência de 50 Hz e resolução de 16 bits, permite controlar o ângulo do servo com granularidade de aproximadamente 0,0055° por unidade de duty cycle, muito além da precisão mecânica do servo (tipicamente ±2°). A separação de alimentação entre o servo (5 V) e o ESP32-S3 (3,3 V) é essencial para evitar surtos de corrente no barramento de alimentação durante o movimento do motor.

4.3.4 Interrupções GPIO para Botões

Os botões de chamada e emergência são tratados exclusivamente por interrupções de hardware (ISR — *Interrupt Service Routines*), configuradas no ESP-IDF via `gpio_install_isr_service()` e `gpio_isr_handler_add()`. Esta abordagem garante que o acionamento dos botões seja capturado independentemente do estado de qualquer tarefa FreeRTOS, seja ela bloqueada, em espera de I/O ou executando

processamento pesado. O debounce é implementado por software na ISR, ignorando transições adicionais por uma janela de 50 ms após o primeiro evento detectado. As ISRs são intencionalmente mínimas: apenas sinalizam uma fila FreeRTOS e retornam imediatamente, deixando o processamento para as tarefas de maior prioridade.

Capítulo 5

Diagrama de Blocos do Sistema

5.1 Diagrama de Blocos de Hardware

A Figura 5.1 apresenta o diagrama de blocos do hardware do sistema VitalKare. O ESP32-S3 ocupa a posição central, recebendo sinais de todos os sensores e periféricos de entrada e gerando sinais de controle para os atuadores e interfaces de saída.

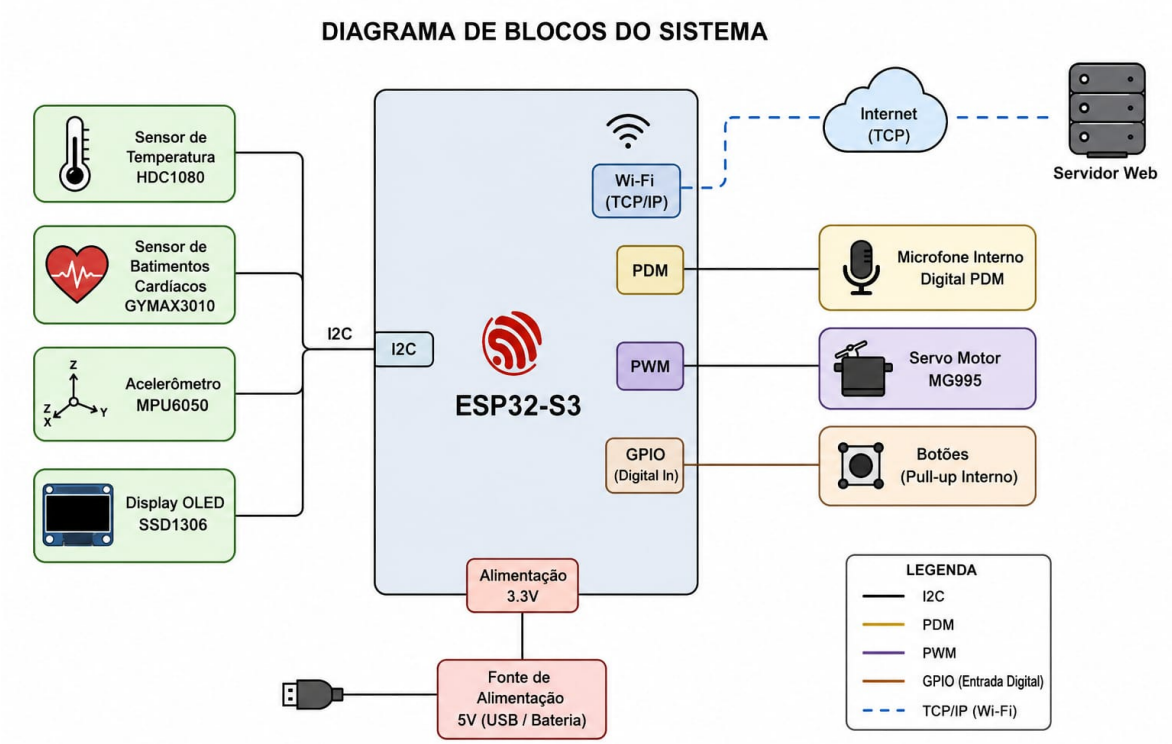


Figura 5.1: Diagrama de blocos do sistema VitalKare.

5.2 Diagrama de Blocos de Software

O firmware é organizado em blocos funcionais independentes executados como tarefas FreeRTOS, conforme ilustra a Figura 5.2.

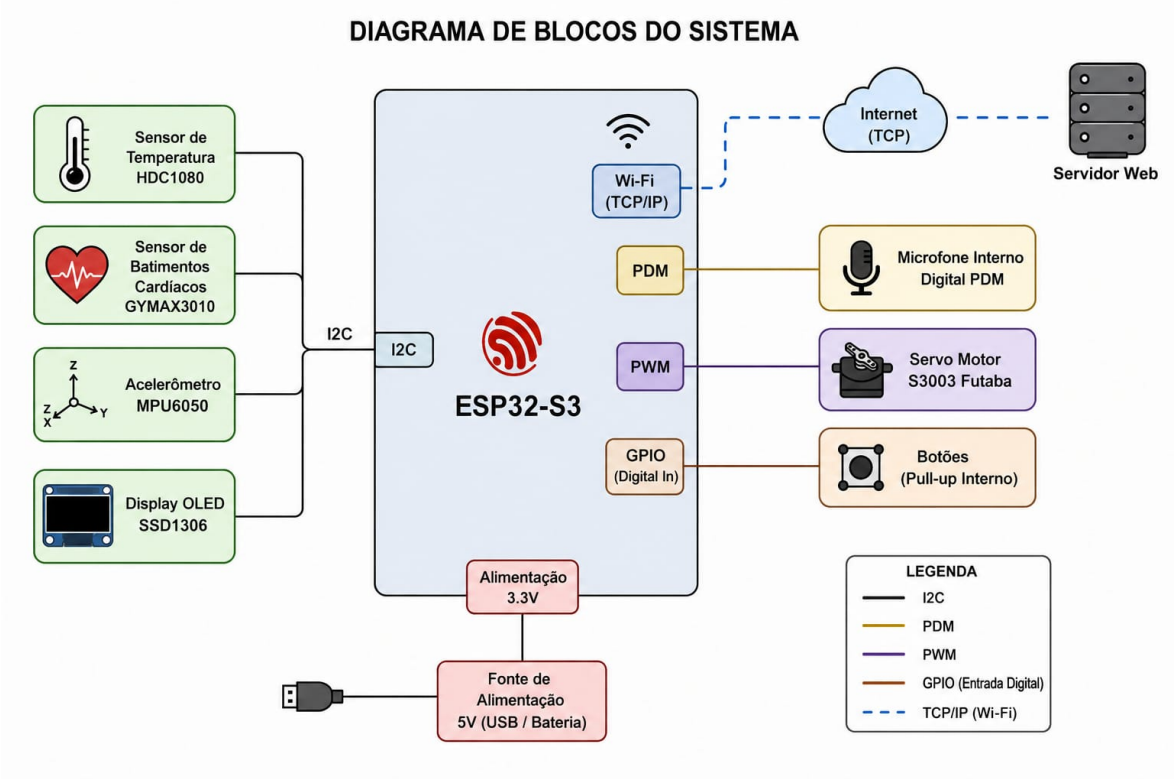


Figura 5.2: Organização em tarefas FreeRTOS do firmware VitalKare. As setas indicam fluxo de dados via filas e eventos.

Capítulo 6

Arquitetura de Firmware

6.1 Modelo de Execução: FreeRTOS

O firmware do VitalKare é construído sobre o **FreeRTOS** (versão integrada ao ESP-IDF 5.x), um sistema operacional de tempo real que oferece escalonamento preemptivo por prioridades, mecanismos de sincronização inter-task (filas, semáforos, grupos de eventos) e gerenciamento de memória estática e dinâmica. A escolha do FreeRTOS é mandatória no contexto ESP-IDF e é particularmente adequada ao domínio hospitalar por garantir que tarefas de alta prioridade — como o tratamento de botões de emergência — recebam CPU em tempo determinístico, independentemente das demais tarefas do sistema.

6.2 Hierarquia de Tarefas

O firmware é organizado em tarefas independentes com prioridades distintas, conforme a Tabela 6.1:

Tabela 6.1: Tarefas FreeRTOS do firmware VitalKare.

Tarefa	Prioridade	Stack (KB)	Core	Descrição
`network_task`	6	8	Core 0	Gerencia socket TCP, envio de telemetria, upload de áudio e heartbeat
`audio_task`	5	16	Core 0	Gerencia gravação I2S/PDM, fecha buffer e enfileira para upload
`sensor_task`	4	4	Core 1	Lê HDC1080, MAX30102, MPU6050; enfileira dados de telemetria
`servo_task`	3	4	Core 1	Lê ângulo MPU6050, calcula duty cycle PWM, controla servo
`display_task`	2	6	Core 1	Renderiza telas no SSD1306, consome dados de telemetria
`watchdog_task`	1	2	Core 0	Alimenta IWDT, monitora saúde das tarefas, registra heartbeat

6.3 Tratamento de Interrupções (ISR)

Os botões de chamada e emergência são o subsistema de maior prioridade absoluta no firmware. A configuração dos pinos como entrada com interrupção de hardware é realizada da seguinte forma:

```
static void IRAM_ATTR button_a_isr_handler(void *arg) {
    /* ISR do botao A - Chamada Simples */
    static uint32_t last_time = 0;
    uint32_t now = esp_timer_get_time() / 1000;
    if ((now - last_time) > 50) {    /* debounce 50 ms */
        last_time = now;
```

```
        event_t evt = { .type = EVT_CALL_BUTTON, .priority =
            PRIORITY_NORMAL };
        BaseType_t higher_prio_woken = pdFALSE;
        xQueueSendFromISR(g_event_queue, &evt, &higher_prio_woken)
            ;
        portYIELD_FROM_ISR(higher_prio_woken);
    }
}

static void IRAM_ATTR button_b_isr_handler(void *arg) {
    /* ISR do botao B - Emergencia com Audio */
    static uint32_t last_time = 0;
    uint32_t now = esp_timer_get_time() / 1000;
    if ((now - last_time) > 50) {
        last_time = now;
        bool pressed = !gpio_get_level(GPIO_BTN_EMERGENCY);
        event_t evt = {
            .type     = pressed ? EVT_EMERGENCY_PRESS :
                EVT_EMERGENCY_RELEASE,
            .priority = PRIORITY_EMERGENCY
        };
        BaseType_t higher_prio_woken = pdFALSE;
        xQueueSendFromISR(g_event_queue, &evt, &higher_prio_woken)
            ;
        portYIELD_FROM_ISR(higher_prio_woken);
    }
}
```

Listing 6.1: Configuração de ISR para botões de chamada e emergência.

A diretiva `IRAM_ATTR` garante que a ISR seja alocada na RAM interna do ESP32-S3, permitindo execução mesmo quando o cache de flash estiver ocupado com outras operações. A chamada a `portYIELD_FROM_ISR()` ao final da ISR permite ao escalonador do FreeRTOS verificar imediatamente se uma tarefa de prioridade mais alta foi desbloqueada pela mensagem enviada à fila.

6.4 Pipeline de Áudio

O pipeline de gravação e transmissão de áudio é um dos subsistemas mais críticos do firmware, por envolver operações de alta largura de banda (16 kHz × 16 bit

= 256 kbps) em tempo real. A Figura 6.1 ilustra o fluxo completo:

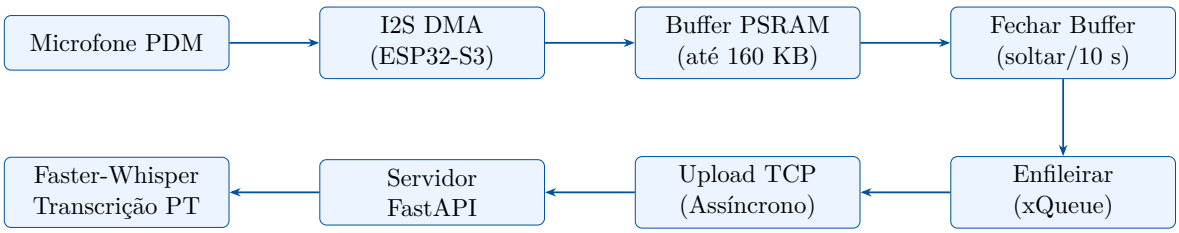


Figura 6.1: Pipeline completo de captura, gravação, transmissão e transcrição de áudio de emergência.

O buffer de áudio é alocado estaticamente em PSRAM durante a inicialização do firmware, garantindo disponibilidade imediata no momento da emergência sem risco de falha de alocação dinâmica. O driver I2S utiliza DMA de duplo buffer (*ping-pong*), de forma que enquanto um buffer DMA é preenchido pelo hardware I2S, o firmware copia o outro para a PSRAM, sem perda de amostras.

6.5 Controle de Posicionamento (MPU6050 + Servo)

O ângulo de inclinação do dispositivo é calculado a partir dos valores brutos do acelerômetro do MPU6050, usando a função arco-tangente aplicada sobre os eixos relevantes. A equação utilizada é:

$$\theta = \arctan\left(\frac{a_X}{\sqrt{a_Y^2 + a_Z^2}}\right) \times \frac{180\check{r}}{\pi} \tag{6.1}$$

onde a_X, a_Y e a_Z são as acelerações medidas nos respectivos eixos (em unidades de g). O ângulo θ calculado é então mapeado linearmente para o range de duty cycle do servo:

$$D = D_{\min} + \frac{(\theta + 90\check{r})}{180\check{r}} \times (D_{\max} - D_{\min}) \tag{6.2}$$

onde $D_{\min} = 1640$ e $D_{\max} = 8190$ correspondem aos pulsos de 500 µs e 2500 µs na resolução de 16 bits a 50 Hz, respectivamente.

6.6 Watchdog e Confiabilidade

O firmware implementa um watchdog de hardware (*Interrupt Watchdog Timer* — IWDT) com período de 5 segundos. A tarefa `watchdog_task` é responsável por alimentar o IWDT periodicamente e por monitorar o health score de cada tarefa principal via *task notification*. Se qualquer tarefa crítica deixar de enviar seu sinal de vida dentro do período esperado, o watchdog registra o evento em log persistente (SD Card) e força um reinício suave (software reset) do sistema, garantindo retorno automático à operação normal. O uso de NVS para armazenar a contagem de resets permite análise posterior de estabilidade.

Capítulo 7

Especificação do Protocolo de Comunicação e IoT

7.1 Avaliação e Seleção de Protocolo

7.1.1 MQTT — Avaliação e Descarte como Protocolo Primário

O protocolo MQTT (*Message Queuing Telemetry Transport*) foi rigorosamente avaliado como candidato primário de comunicação para o VitalKare, dada sua ampla adoção no ecossistema IoT hospitalar e suas características de QoS (Quality of Service) com garantia de entrega (QoS 1 e 2). O README original do projeto e a versão anterior do sistema (baseado em BitDogLab/RP2040) utilizavam MQTT como protocolo principal.

Entretanto, após análise arquitetural detalhada das necessidades específicas do VitalKare, identificou-se que o modelo de publicação/assinatura do MQTT introduz complexidades desnecessárias para o ambiente de LAN hospitalar controlado em que o sistema opera. Os principais fatores que levaram ao descarte do MQTT como protocolo primário foram:

1. **Sobrecarga de Infraestrutura:** MQTT requer um broker intermediário (Mosquitto, EMQX ou similar) que representa um ponto adicional de falha na arquitetura. Em um ambiente hospitalar, a disponibilidade do sistema é crítica, e cada componente adicional aumenta o risco de indisponibilidade.
2. **Latência Adicional:** O modelo publish/subscribe introduz latência de dupla hop (dispositivo → broker → servidor), que, embora tipicamente inferior a 10 ms em LAN, é desnecessária quando o destino dos dados é sempre o mesmo servidor FastAPI.
3. **Overhead de Protocolo:** O cabeçalho MQTT acrescenta 2 a 12 bytes por mensagem, além do handshake CONNECT/CONNACK na sessão inicial.

Para mensagens de telemetria de alta frequência, este overhead se acumula significativamente.

4. **Complexidade de QoS para Áudio:** O upload de buffers de áudio (até 160 KB) via MQTT com QoS 2 exigiria fragmentação em múltiplos pacotes PUBLISH com controle de QoS individual, implementação significativamente mais complexa do que um único write TCP com controle de sequenciamento próprio.
5. **Ambiente Controlado:** O sistema opera em LAN hospitalar privada com infraestrutura Wi-Fi dedicada e gerenciada. As garantias de entrega e roteamento dinâmico do MQTT são projetadas para redes instáveis e heterogêneas — características não presentes no ambiente de implantação do VitalKare.

7.1.2 TCP — Protocolo Selecionado

TCP persistente com framing binário foi selecionado como o protocolo primário de comunicação do VitalKare pelos seguintes motivos técnicos:

- **Comunicação direta:** Socket TCP direto entre o ESP32-S3 e o servidor FastAPI, sem intermediários. A eliminação do broker reduz a latência e os pontos de falha.
- **Menor overhead:** O cabeçalho TCP (20 bytes) é fixo e independente do tamanho do payload, ao contrário do MQTT que adiciona overhead variável por mensagem.
- **Sockets persistentes:** A conexão TCP é estabelecida uma única vez na inicialização e mantida pelo ciclo de vida do dispositivo, eliminando o custo de handshake por mensagem.
- **Controle total do protocolo de aplicação:** O framing binário proprietário permite otimizações específicas ao domínio (campos de prioridade, tipo de evento, sequenciamento) impossíveis com o protocolo MQTT padrão.
- **Upload de áudio eficiente:** Buffers de áudio de até 160 KB são enviados como um único fluxo TCP contíguo, com controle de fluxo nativo do TCP, sem fragmentação em nível de aplicação.
- **Determinismo:** Em LAN hospitalar controlada, a entrega TCP é virtual-

mente garantida sem as complexidades adicionais do QoS MQTT.

7.2 Arquitetura TCP

7.2.1 Modelo de Conexão

O ESP32-S3 opera como cliente TCP. Uma única conexão é mantida ao servidor FastAPI na porta 8001. O servidor mantém uma tabela de conexões ativas, indexada por `device_id`, permitindo identificar qual leito é a origem de cada pacote.

O ciclo de vida da conexão segue o modelo ilustrado abaixo:

1. **Inicialização:** Após estabelecer conexão Wi-Fi, a `network_task` inicia a tentativa de conexão TCP ao servidor.
2. **Handshake de Aplicação:** O ESP32-S3 envia um pacote `HELLO` com `device_id`, `ward_id`, versão de firmware e capacidades do hardware.
3. **Operação Normal:** Telemetria periódica, alertas de chamada/emergência e uploads de áudio são enviados conforme gerados.
4. **Heartbeat:** A cada 30 segundos, o dispositivo envia um pacote `HEARTBEAT` com RSSI, heap livre e uptime. O servidor considera o leito *offline* após 90 segundos sem heartbeat.
5. **Reconexão:** Em caso de desconexão, a `network_task` implementa reconexão com backoff exponencial: 1 s, 2 s, 4 s, 8 s, ..., máximo de 60 s.

7.2.2 Framing de Pacotes

Cada pacote TCP é precedido por um cabeçalho binário de 8 bytes que permite ao servidor desserializar corretamente o payload, conforme a Tabela 7.1:

Tabela 7.1: Estrutura do cabeçalho de pacote TCP (framing binário).

Offset	Tamanho	Campo	Descrição
0	1 byte	`type`	Tipo do pacote (enumeração)
1	1 byte	`priority`	Prioridade: 0=normal, 1=alta, 2=emergência
2	2 bytes	`seq`	Número de sequência (uint16, incrementado)
4	4 bytes	`length`	Comprimento do payload em bytes (uint32, big-endian)
8+	N bytes	`payload`	Dados JSON (telemetria/alertas) ou PCM raw (áudio)

7.2.3 Estratégia de ACK/NACK

Para pacotes de alta prioridade (chamadas e emergências), o servidor retorna um pacote `ACK` com o número de sequência correspondente. O firmware mantém uma fila de pacotes não confirmados e retransmite após timeout de 5 segundos, com até 3 tentativas. Para telemetria de baixa prioridade (dados de sensores), nenhum ACK é aguardado — a perda de uma amostra é tolerável dado o fluxo contínuo.

7.3 Estrutura de Dados — Payloads JSON

7.3.1 Pacote de Telemetria

```
{
  "type": "telemetry",
  "device_id": "leito_101",
  "ward_id": "UTI_A",
  "timestamp": "2026-05-19T14:32:00.123Z",
  "firmware_version": "1.4.2",
  "vitals": {
    "heart_rate_bpm": 72,
    "spo2_percent": 98.5,
    "temperature_celsius": 22.4,
    "humidity_percent": 55.1
  },
  "system": {
```

```
    "wifi_rssi_dbm": -52,
    "free_heap_bytes": 185432,
    "uptime_seconds": 3842,
    "servo_angle_deg": 15.3
  }
}
```

Listing 7.1: Payload JSON de telemetria de sinais vitais (enviado a cada 5 s).

7.3.2 Pacote de Chamada Simples (com Áudio)

Importante: A Chamada Simples (botão A) também inicia uma gravação de áudio de curta duração (até 5 segundos). Esta decisão arquitetural garante que, mesmo em chamadas de rotina, o contexto sonoro do quarto seja capturado, auxiliando na triagem do tipo de assistência necessária.

```
{
  "type": "call_request",
  "device_id": "leito_101",
  "ward_id": "UTI_A",
  "timestamp": "2026-05-19T14:33:15.456Z",
  "call": {
    "button_type": "SIMPLE_CALL",
    "priority": "NORMAL",
    "duration_pressed_ms": 320,
    "audio": {
      "recording": true,
      "duration_seconds": 3.2,
      "sample_rate_hz": 16000,
      "format": "PCM_16BIT_MONO",
      "upload_pending": true,
      "audio_ref_id": "leito_101_call_20260519_143315"
    }
  }
}
```

Listing 7.2: Payload JSON de chamada simples com metadados de áudio.

7.3.3 Pacote de Emergência

```json
{
  "type": "emergency_alert",
  "device_id": "leito_101",
  "ward_id": "UTI_A",
  "timestamp": "2026-05-19T14:35:02.789Z",
  "emergency": {
    "button_type": "EMERGENCY",
    "priority": "CRITICAL",
    "escalation_level": 3,
    "duration_pressed_ms": 8400,
    "audio": {
      "recording": true,
      "duration_seconds": 8.4,
      "sample_rate_hz": 16000,
      "format": "PCM_16BIT_MONO",
      "upload_pending": true,
      "audio_ref_id": "leito_101_emerg_20260519_143502",
      "transcription_requested": true,
      "transcription_language": "pt-BR"
    },
    "vitals_at_event": {
      "heart_rate_bpm": 118,
      "spo2_percent": 94.2,
      "temperature_celsius": 22.4
    }
  }
}
```

Listing 7.3: Payload JSON de emergência com metadados de áudio e escalada de prioridade.

7.4 Fluxo de Áudio — Detalhamento

O fluxo de captura, transmissão e transcrição de áudio é descrito passo a passo abaixo:

1. **Início de gravação:** A ISR do botão B (emergência) detecta a borda de descida (pressão) e sinaliza a `audio_task` via fila de alta prioridade com o evento `EVT_EMERGENCY_PRESS`.

2. **Captura I2S/PDM:** A `audio_task` habilita o driver I2S PDM e inicia a captura DMA. Amostras PCM 16-bit a 16 kHz são armazenadas sequencialmente no buffer alocado em PSRAM.
3. **Monitoramento de limite:** Um timer de software é iniciado com período de 10 segundos. Se o botão não for solto antes, o timer dispara e sinaliza o encerramento da gravação.
4. **Encerramento da gravação:** Ao detectar a borda de subida do botão B (soltura) ou ao expirar o timer de 10 segundos, a `audio_task` para o driver I2S, fecha o buffer com o tamanho real capturado e desabilita o microfone para economizar energia.
5. **Enfileiramento para upload:** O buffer fechado é enfileirado na `audio_upload_queue`, que é processada assincronamente pela `network_task`. Esta separação garante que a telemetria continue sendo enviada normalmente durante o upload do áudio.
6. **Upload TCP:** A `network_task` envia o cabeçalho de pacote de áudio seguido do payload PCM bruto. O servidor FastAPI recebe o stream via TCP, detecta o fim do pacote pelo campo `length` do cabeçalho, e armazena o áudio em MinIO com o `audio_ref_id` referenciado no pacote de alerta.
7. **Transcrição assíncrona:** O backend enfileira uma tarefa de transcrição no worker Faster-Whisper (modelo `small`, PT-BR). A transcrição resultante é persistida no PostgreSQL e propagada ao dashboard via WebSocket em tempo real.
8. **Exibição no dashboard:** O componente `EmergencyBanner` do dashboard React exibe o alerta em modo de destaque máximo, com a transcrição de áudio aparecendo progressivamente conforme o modelo Whisper processa os segmentos de fala.

Garantia de isolamento de pipeline: O upload de áudio nunca bloqueia o envio de telemetria ou de novos alertas. A fila de upload de áudio é processada em segundo plano, garantindo que a telemetria clínica contínua mantenha sua periodicidade mesmo durante transferências de arquivos de áudio.

Capítulo 8

Dashboard e Servidor

8.1 Arquitetura do Backend

O servidor central do VitalKare é desenvolvido em **FastAPI** (Python 3.11), um framework ASGI de alto desempenho que suporta nativamente programação assíncrona via `asyncio`. Esta característica é essencial para gerenciar simultaneamente centenas de conexões WebSocket de clientes de dashboard e múltiplas conexões TCP persistentes de dispositivos embarcados.

A arquitetura do backend é composta pelos seguintes subsistemas:

TCP Server (porta 8001): Servidor TCP assíncrono que mantém conexões persistentes com cada dispositivo KIIRA V2. Cada conexão é gerenciada por uma coroutine Python dedicada, responsável por desserializar pacotes, persistir dados e emitir eventos ao Event Processor.

Event Processor: Componente central de processamento de eventos, que recebe eventos normalizados do TCP Server (telemetria, chamadas, emergências, áudio), persiste no banco de dados e propaga ao WebSocket Handler.

WebSocket Handler (porta 8000/ws): Gerencia conexões WebSocket com os clientes do dashboard React. Ao receber um novo evento do Event Processor, transmite a atualização para todos os clientes conectados (broadcast) ou apenas para o cliente responsável pela ala correspondente.

Heartbeat Monitor: Tarefa assíncrona que verifica a cada 30 segundos se todos os dispositivos registrados enviaram seu heartbeat dentro do limiar de 90 segundos. Dispositivos sem heartbeat são marcados como *offline* no banco de dados e o dashboard é notificado.

Transcription Worker: Fila de tarefas de transcrição processadas pelo modelo Faster-Whisper (self-hosted). Cada áudio recebido é submetido à fila; a transcrição resultante é associada ao evento de origem e transmitida ao dashboard.

8.2 Camada de Dados

O banco de dados principal utiliza **PostgreSQL 16** com a extensão **TimescaleDB**, que converte tabelas de séries temporais em hypertables otimizadas para inserção e consulta de dados sequenciais de alta frequência. As principais entidades de dados são:

Tabela 8.1: Principais entidades de dados do sistema VitalKare.

Tabela/Entidade	Descrição
`beds`	Cadastro de leitos (device_id, ward_id, status, paciente atual)
`telemetry`	Séries temporais de sinais vitais (TimescaleDB hypertable)
`events`	Log de eventos (chamadas, emergências, conexões, desconexões)
`audio_records`	Metadados de gravações de áudio (ref MinIO, transcrição)
`patients`	Cadastro de pacientes atribuídos a leitos
`nurses`	Cadastro de profissionais de enfermagem (RBAC)
`alerts`	Alertas gerados e histórico de atendimento

8.3 Dashboard React 18

O dashboard de enfermagem é desenvolvido em **React 18 + TypeScript + Tailwind CSS**, com gerenciamento de estado via **Zustand**. A comunicação em tempo real é realizada via WebSocket com auto-reconexão.

A interface é organizada nos seguintes componentes principais:

BedGrid: Grade de leitos da ala, cada leito representado por um `BedCard` com indicadores de status (normal, chamada, emergência, offline), BPM, SpO2 e temperatura em tempo real.

BedCard: Cartão individual de leito com indicadores visuais codificados por cor: verde = normal; âmbar = chamada simples; **VERMELHO PULSANTE** = emergência ativa. Inclui sparklines de BPM e temperatura dos últimos 5 minutos.

EmergencyBanner: Banner de tela cheia com alto contraste vermelho exibido durante emergências ativas. Inclui identificação do leito, sinais vitais no momento do evento, player de áudio e transcrição de voz em tempo real.

EventLog: Painel lateral com histórico cronológico de eventos de todos os leitos, com

filtros por tipo (telemetria, chamada, emergência) e por leito.

8.4 Interação Operacional da Equipe de Enfermagem

O fluxo operacional do dashboard durante uma emergência é o seguinte:

1. O paciente pressiona o botão de emergência no dispositivo VitalKare ao lado do leito.
2. Em menos de 500 ms, o `EmergencyBanner` é exibido em todas as estações de dashboard do posto de enfermagem.
3. A transcrição de áudio é exibida progressivamente à medida que o Faster-Whisper processa os segmentos (latência típica de 2–4 segundos para 10 s de áudio no modelo `small`).
4. O profissional de enfermagem visualiza a gravidade, o leito afetado e a transcrição antes de se deslocar, podendo mobilizar os recursos adequados.
5. Ao chegar ao quarto e atender o chamado, o profissional marca o alerta como atendido no dashboard via botão de confirmação.
6. O ESP32-S3 do leito recebe um pacote `ACK` do servidor, exibindo no OLED a mensagem de confirmação de atendimento e retornando ao estado de monitoramento normal.

Capítulo 9

Modelagem 3D dos Gabinetes

9.1 Gabinete Principal — Beira do Leito

O gabinete principal tem a função de abrigar a placa KIIRA V2 (ESP32-S3), protegendo os componentes eletrônicos do ambiente hospitalar (umidade, produtos de limpeza, choques mecânicos acidentais) enquanto mantém acessibilidade às interfaces necessárias para operação.

9.1.1 Especificações de Design

Tabela 9.1: Requisitos de design do gabinete principal VitalKare.

Requisito	Especificação
Material	PLA ou PETG (impressão FDM); ABS apenas se necessário para temperatura
Dimensões aprox.	120 mm (L) × 80 mm (A) × 40 mm (P)
Abertura display	Recorte frontal 30 mm × 13 mm para tela OLED 0,96"
Abertura microfone	Grade circular com furos de 1,5 mm de diâmetro (mínimo 9 furos)
Passagem de cabos	Calha lateral com prensa-cabo para cabo USB-C (alimentação)
Botões externos	Dois conectores de 2 pinos para cabos de extensão de botões
Fixação	2 abas com furo de 4 mm para parafuso ou velcro industrial
Ventilação	Fendas laterais de 1 mm para dissipação de calor
Acabamento	Superfície externa lisa; impressão em branco ou azul hospitalar
Certificação	Design compatível com requisitos de limpeza hospitalar

9.1.2 Posicionamento do Acelerômetro

O MPU6050 deve ser posicionado no gabinete de forma que seu eixo Z esteja perpendicular ao plano do colchão quando o gabinete é fixado lateralmente ao leito.

Esta orientação garante que a leitura de ângulo reflete com precisão a inclinação real do leito. Marcações físicas no gabinete indicam a orientação correta de montagem.

[Placeholder — Inserir render 3D do gabinete principal]
Vista frontal, lateral e posterior do gabinete KIIRA V2.
Ferramentas sugeridas: FreeCAD, Fusion 360 ou SolidWorks.

Figura 9.1: Render 3D do gabinete principal VitalKare (a ser inserido após modelagem CAD).

9.2 Acessório Sensor Cardíaco — Oxímetro de Dedo

O acessório de oximetria de pulso é uma peça ergonômica independente que acomoda o sensor MAX30102/MAX30105 em formato de clip hospitalare, conectado ao gabinete principal via cabo flexível de 3 pinos (VCC, GND, I2C).

9.2.1 Requisitos de Design

Tabela 9.2: Requisitos de design do acessório de oxímetria de dedo.

Requisito	Especificação
Formato	Clip tipo pinça, similar a oxímetro hospitalar comercial
Faixa de dedo	Adaptável a dedos de 10 mm a 22 mm de diâmetro
Mola de pressão	Mola embutida no design (geometria de mola de lâmina impressa)
Alojamento do sensor	Cavidade exata para PCB do MAX30102 (5 mm × 8 mm)
Isolamento óptico	Paredes internas opacas (preto ou cinza escuro) para bloquear luz ambiente
Janela óptica	Abertura mínima de 3 mm × 3 mm sobre o fotodetector, sem vidro
Material	PETG preto (rigidez e opacidade) + TPU na parte de contato com pele
Cabo de saída	Guia-cabo integrado para cabo de 4 vias (V, G, SDA, SCL)
Identificação	Logotipo VitalKare em relevo na face externa

O design do acessório é crítico para a qualidade das leituras do sensor MAX30102: a literatura técnica do componente indica que a supressão de luz ambiente é o principal fator de ruído nas leituras de fotopletismografia (PPG). A geometria de encapsulamento deve garantir que os LEDs vermelho e infravermelho do sensor iluminem exclusivamente o tecido do dedo, sem reflexão direta ao fotodetector (reflexão especular parasita).



Figura 9.2: Render 3D do acessório de oxímetria de pulso VitalKare (a ser inserido após modelagem CAD).

9.3 Acoplamento Mecânico do Servo Motor

O projeto prevê um terceiro elemento de modelagem: a peça de acoplamento entre o servo motor e a manivela da cama hospitalar. Este adaptador deve ser projetado especificamente para o modelo de cama hospitalar alvo, mas os princípios gerais de design são:

- O eixo do servo deve se alinhar coaxialmente ao eixo da manivela;
- Um acoplamento por interferência ou por parafuso de pressão garante transmissão de torque adequada;
- O adaptador deve incluir uma chave ou fenda de escape que permita operação manual em caso de falha do servo ou do sistema embarcado;
- O torque máximo do servo (2,5 kg · cm a 5 V) deve ser verificado contra o torque necessário para mover a cama específica do hospital-alvo.

Capítulo 10

Viabilidade Técnica

10.1 Viabilidade de Hardware

Os componentes selecionados para o VitalKare são todos comercialmente disponíveis, amplamente utilizados no ecossistema ESP32 e suportados por drivers estáveis no ESP-IDF 5.x. O custo estimado do hardware por leito é significativamente inferior ao de soluções hospitalares proprietárias, com potencial de implantação em larga escala. A plataforma KIIRA V2, baseada no ESP32-S3, é especificamente projetada para projetos de residência EmbarcaTech e já inclui a maioria dos periféricos necessários integrados.

10.2 Viabilidade de Firmware

10.2.1 Capacidade de Memória

Tabela 10.1: Análise de utilização de memória do firmware VitalKare.

Componente	RAM Usada	Flash Usada	Observação
FreeRTOS Kernel	~12 KB	—	Kernel + scheduler
Stack de Tarefas	~40 KB	—	6 tarefas × ~6 KB médio
Wi-Fi Stack (LwIP)	~60 KB	—	Heap dinâmico Wi-Fi
Buffer de Áudio (PSRAM)	160 KB (PSRAM)	—	Alocado em PSRAM 8 MB
Drivers I2C/I2S/PWM	~8 KB	—	Buffers DMA
Código do Firmware	—	~800 KB	Estimado para produção
RAM Interna Total	~120 KB	—	De 520 KB disponíveis
Margem Livre	~400 KB	—	Headroom confortável

O uso de PSRAM externa de 8 MB para o buffer de áudio é o fator decisivo que torna o pipeline de áudio viável no ESP32-S3. Sem PSRAM, o buffer de 160 KB consumiria 31% da RAM interna disponível, deixando margem insuficiente para a pilha TCP/IP e os buffers de tarefas.

10.2.2 Requisitos de Tempo Real

Os requisitos de tempo real do sistema foram analisados quanto à viabilidade com FreeRTOS:

- **Latência de botão:** Requisito < 10 ms. Alcançado via ISR de hardware (~1 ms).
- **Taxa de telemetria:** 1 sample/5 s. Trivialmente alcançável com qualquer configuração de tarefas.
- **Latência de áudio:** O DMA I2S opera em modo contínuo sem gaps, garantindo captura sem perda de amostras.
- **Controle de servo:** 10 Hz de atualização de ângulo é mais que suficiente

para controle de posicionamento de leito.

10.3 Viabilidade de Rede

A comunicação TCP sobre Wi-Fi 802.11n em ambiente de LAN hospitalar controlada apresenta os seguintes parâmetros de viabilidade:

Tabela 10.2: Análise de largura de banda de rede do sistema VitalKare.

Tipo de Tráfego	Taxa Média	Observação
Telemetria (JSON)	~200 B / 5 s = 40 B/s	Por leito
Heartbeat	~100 B / 30 s	Por leito
Upload de áudio	160 KB / evento = ~128 kbps	Burst; assíncrono
Total por leito (steady state)	< 1 kbps	Sem eventos de áudio
100 leitos simultâneos	< 100 kbps + bursts de áudio	Viável em Wi-Fi 802.11n

A largura de banda de telemetria por leito em estado estacionário é desprezível (< 1 kbps). Mesmo com 100 leitos simultâneos e uploads de áudio periódicos, o sistema permanece dentro das capacidades típicas de uma infraestrutura Wi-Fi hospitalar corporativa.

10.4 Viabilidade de Escalabilidade

A arquitetura do VitalKare foi projetada para escalar de um protótipo de leito único para múltiplas alas hospitalares com mínimas alterações:

- O servidor FastAPI é stateless entre requisições e escala horizontalmente com múltiplas instâncias atrás de um load balancer;
- O PostgreSQL/TimescaleDB escala verticalmente (hardware) e horizontalmente (read replicas) para alta volumetria de dados de séries temporais;
- O MinIO é nativamente distribuído e escala para petabytes de armazenamento de mídia;

- O provisionamento de novos dispositivos é automatizado via script Python (`provision.py`), que configura SSID, senha, IP do servidor e device_id via NVS sem necessidade de recompilação do firmware.

10.5 Riscos Técnicos e Mitigações

Tabela 10.3: Matriz de riscos técnicos e estratégias de mitigação.

Risco	Impacto	Mitigação
Perda de conexão Wi-Fi	Alto	Reconexão automática com backoff exponencial; cache offline em SD Card
Falha do sensor MAX30102	Médio	Retry I2C automático; flag de indisponibilidade na telemetria
Travamento de tarefa	Alto	Watchdog IWDT + monitoramento de health por notificação FreeRTOS
Buffer de áudio cheio	Médio	Limite estrito de 10 s; liberação imediata após upload; PSRAM dedicada
Queda do servidor	Alto	Cache offline em SD Card; upload retroativo ao reconectar
Ruído em leitura de BPM	Médio	Filtro de média móvel de 5 amostras; descarte de outliers (HR < 40 ou > 220)

Capítulo 11

Conclusão

Este documento formaliza a arquitetura completa do sistema VitalKare — uma plataforma embarcada de monitoramento hospitalar à beira do leito desenvolvida como projeto de encerramento da Residência Tecnológica EmbarcaTech em Sistemas Embarcados. A análise técnica apresentada ao longo dos capítulos anteriores demonstra que o projeto é tecnicamente viável, arquiteturalmente sólido e clinicamente relevante.

A escolha do ESP32-S3 (KIIRA V2) como microcontrolador central mostrou-se acertada: o chip oferece recursos de hardware suficientes para todos os periféricos requeridos (I2C, I2S, PWM, GPIO com ISR) em um único pacote, com suporte SDK maduro (ESP-IDF 5.x + FreeRTOS) e comunidade ativa. A PSRAM externa de 8 MB é o fator habilitador crítico para o pipeline de áudio, eliminando a principal restrição de memória do projeto.

A decisão arquitetural de adotar **TCP persistente** em substituição ao MQTT como protocolo primário de comunicação representa uma escolha madura e fundamentada nas características específicas do ambiente hospitalar: LAN controlada, destino único de dados, necessidade de upload eficiente de buffers de áudio e eliminação de ponto único de falha (broker). Esta decisão simplifica a infraestrutura de implantação sem sacrificar nenhum requisito funcional do sistema.

O pipeline de áudio de emergência — desde a pressão do botão até a exibição da transcrição no dashboard — é o diferencial técnico central do VitalKare. A integração de captura I2S/PDM em PSRAM, upload TCP assíncrono e transcrição por Faster-Whisper cria um fluxo de informação clínica contextual que transforma fundamentalmente a qualidade da resposta da equipe de enfermagem a emergências hospitalares.

11.1 Trabalhos Futuros

Como extensões naturais do sistema VitalKare, identificam-se as seguintes oportunidades de desenvolvimento futuro:

1. **Câmera OV5640:** A plataforma KIIRA V2 suporta câmera OV5640 via interface MIPI/DVP. A integração de captura de imagem sob demanda permitiria verificação visual remota do leito pelo enfermeiro sem deslocamento físico.
2. **Atualização OTA:** Implementação de atualização de firmware over-the-air via HTTPS, permitindo atualizações em frota de dispositivos sem intervenção física.
3. **Detecção de queda:** Uso do MPU6050 para detecção de eventos de aceleração anômalos (queda de paciente) com acionamento automático de alerta de emergência.
4. **Integração com HIS:** Integração com sistemas de informação hospitalar (HIS) existentes via HL7 FHIR, permitindo correlação de alertas com prontuários eletrônicos de pacientes.
5. **Análise de tendência por IA:** Utilização das séries temporais de BPM e SpO2 armazenadas no TimescaleDB para detecção preditiva de deterioração clínica por modelos de machine learning.

11.2 Considerações Finais

O VitalKare demonstra que tecnologias de sistemas embarcados de baixo custo, quando adequadamente arquitetadas e integradas, têm o potencial de transformar workflows hospitalares críticos. A plataforma desenvolvida nesta residência serve como prova de conceito técnica para uma categoria de dispositivos IoT hospitalares que podem reduzir significativamente o tempo de resposta a emergências, melhorar o conforto e a segurança de pacientes acamados e fornecer à equipe de enfermagem informações contextuais ricas antes mesmo de chegarem ao quarto do paciente.

Referências Bibliográficas

[1] ESPRESSIF SYSTEMS. **ESP32-S3 Technical Reference Manual**. Version 1.4. Shanghai: Espressif Systems, 2024. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf. Acesso em: 19 mai. 2026.

[2] ESPRESSIF SYSTEMS. **ESP-IDF Programming Guide — FreeRTOS**. Version 5.x. Disponível em: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/system/freertos.html. Acesso em: 19 mai. 2026.

[3] BARRY, R. **Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide**. Real Time Engineers Ltd., 2016.

[4] TEXAS INSTRUMENTS. **HDC1080 Low Power, High Accuracy Digital Humidity Sensor with Temperature Sensor — Datasheet**. Dallas: Texas Instruments, 2017.

[5] MAXIM INTEGRATED. **MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health — Datasheet**. San Jose: Maxim Integrated, 2018.

[6] INVENSENSE (TDK). **MPU-6000/MPU-6050 Product Specification Revision 3.4**. San Jose: InvenSense, 2013.

[7] SOLOMON SYSTECH. **SSD1306 Advance Information — 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller**. Hong Kong: Solomon Systech, 2008.

[8] POSNER, R.; VENESS, C. **Faster Whisper: Faster Inference of OpenAI's Whisper with CTranslate2**. GitHub Repository. Disponível em: https://github.com/SYSTRAN/faster-whisper. Acesso em: 19 mai. 2026.

[9] TIANGOLO, S. **FastAPI — Modern, Fast (High-Performance), Web Framework for Building APIs with Python 3.8+**. Disponível em: https://fastapi.tiangolo.com. Acesso em: 19 mai. 2026.

[10] TIMESCALE, INC. **TimescaleDB Documentation — Open-Source Time-Series SQL Database**. Disponível em: https://docs.timescale.com. Acesso em: 19 mai. 2026.

[11] NATIONAL PRESSURE INJURY ADVISORY PANEL (NPIAP). **Prevention and Treatment of Pressure Ulcers/Injuries: Clinical Practice Guideline**. 3. ed. NPIAP, 2019.

[12] CHEN, M.; MAO, S.; LIU, Y. Big Data: A Survey. **Mobile Networks and Applications**, v. 19, n. 2, p. 171–209, 2014. DOI: 10.1007/s11036-013-0489-0.

[13] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO 13485:2016 — Dispositivos médicos — Sistemas de gestão da qualidade — Requisitos para fins regulatórios**. Rio de Janeiro: ABNT, 2016.

[14] PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH, 2016.

[15] EMBARCATECH. **Residência Tecnológica em Sistemas Embarcados — Material Didático**. Brasília: EmbarcaTech, 2026.

Apêndice A

Estrutura do Repositório

O repositório do projeto VitalKare é organizado conforme a estrutura abaixo, permitindo desenvolvimento paralelo das camadas de firmware, backend e dashboard por diferentes membros da equipe:

```
VitalKare/
|-- firmware/                # ESP-IDF - KIIRA V2 (ESP32-S3)
|   |-- config/              # app_config.h, hardware_config.h
|   |-- main/                # main.c - orquestrador de tasks
|   |-- components/          # Módulos FreeRTOS independentes
|   |   |-- core/            # Maquina de estados global
|   |   |-- events/          # Event dispatcher (FreeRTOS queue)
|   |   |-- network/         # Wi-Fi manager + TCP client
|   |   |-- buttons/         # ISR handlers com debounce
|   |   |-- sensors/         # HDC1080, MPU6050, MAX30102
|   |   |-- display/         # Driver OLED SSD1306
|   |   |-- audio/           # Captura PDM I2S (PSRAM)
|   |   |-- storage/         # SD SDMMC + log offline
|   |   `-- watchdog/        # IWDT + monitoramento
|   `-- tools/
|       `-- provision.py     # Provisionamento NVS
|
|-- backend/                 # FastAPI - Servidor Central
|   `-- app/
|       |-- api/v1/          # REST + WebSocket endpoints
|       |-- core/            # TCP handler, event processor
|       |-- models/          # SQLAlchemy ORM
|       `-- db/              # Sessao async + Alembic
|
|-- dashboard/               # React 18 + TypeScript
|   `-- src/
|       |-- components/      # BedCard, BedGrid, EmergencyBanner
|       |-- hooks/           # useWebSocket
|       `-- store/           # Zustand state management
```

```
|
`-- docker/                  # Docker Compose + infra
    |-- docker-compose.yml
    `-- mosquitto/
```

Listing A.1: Estrutura de diretórios do repositório VitalKare.

Apêndice B

Guia de Provisionamento do Firmware

O provisionamento de credenciais de rede e configuração do dispositivo é realizado via script Python, evitando credenciais hardcoded no código-fonte:

```
# Instalar dependencias
pip install esptool

# Executar o script de provisionamento
python firmware/tools/provision.py --port COM16

# O script solicita interativamente:
#   Wi-Fi SSID:     <SSID da rede hospitalar>
#   Wi-Fi Password: <senha da rede>
#   Server IP:      <IP do servidor FastAPI, ex: 192.168.1.100>
#   Server Port:    <porta TCP, padrao 8001>
#   Device ID:      <identificador unico do leito, ex: leito_101>
#   Ward ID:        <identificador da ala, ex: UTI_A>

# Compilar e flashar o firmware
cd firmware
idf.py set-target esp32s3
idf.py -p COM16 -b 921600 flash monitor
```

Listing B.1: Fluxo de provisionamento do firmware via NVS.